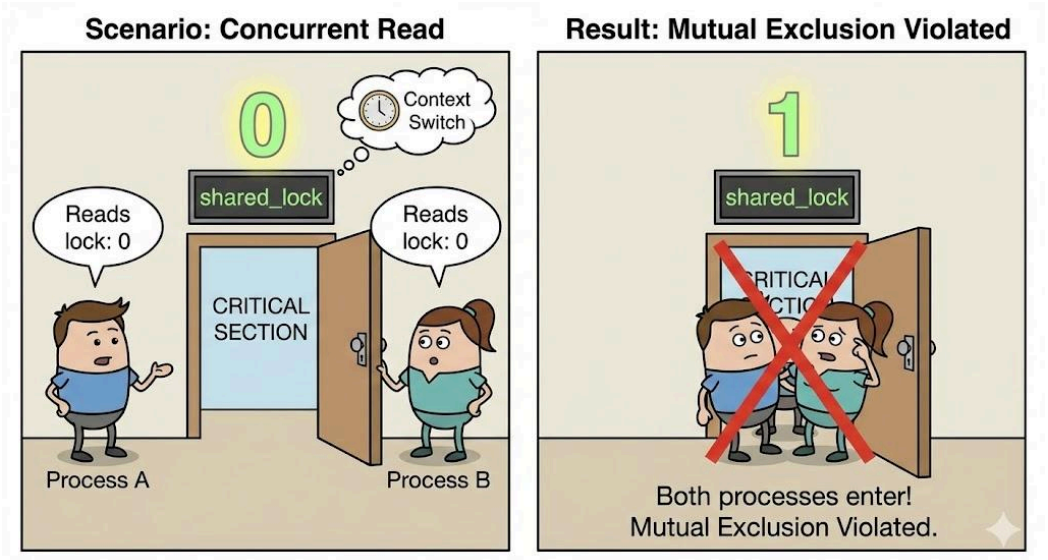
Lecture 11 NOTES – Process Synchronization II: Software-Based Solutions

Simple Software Attempts & Their Failures

Before we had advanced locks, programmers tried to use simple variables to control access. These attempts failed because they didn't account for the **timing** of CPU context switches.

Simple Boolean Lock (The "Unprotected Key")

- **The Idea:**
  - We use a single shared variable called lock.
  - If lock == 0, the door is open. A process enters and sets lock = 1.
  - If lock == 1, the door is closed. The process waits.
  - 

- **The Failure (Race Condition):**
  - The action of "checking the lock" and "setting the lock" is **not atomic** (it's two separate steps).
  - **Scenario:**
    1. Process A reads lock and sees 0.
    2. *Context Switch happens.*
    3. Process B reads lock and also sees 0.
    4. Process B sets lock = 1 and enters.
    5. *Context Switch back to A.*
    6. Process A (remembering the 0) sets lock = 1 and enters too.
- **Violates: Mutual Exclusion** (Both are in the Critical Section at the same time).

Implementation Code:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

// Shared Lock Variable (0 = Free, 1 = Busy) [cite: 92, 93]
int lock = 0;

void* process(void* arg) {
   int id = *(int*)arg;

   // --- ENTRY SECTION ---
   // The "While" loop checks if lock is free
   // Ideally: Wait until lock is 0
   printf("Process %d: Checking lock ... \n", id);
   while (lock != 0);

   // ARTIFICIAL DELAY to force the Race Condition (simulate context
switch)
   // This mimics the "Gap" shown in your slide's timing diagram
[cite: 124]
   sleep(1);

   // Set lock to 1 (Busy)
   lock = 1;
   // ----------------------

   // --- CRITICAL SECTION ---
   printf("Process %d: ENTERED Critical Section!\n", id);
   sleep(2); // Simulate work
   printf("Process %d: LEAVING Critical Section.\n", id);
   // ------------------------
```

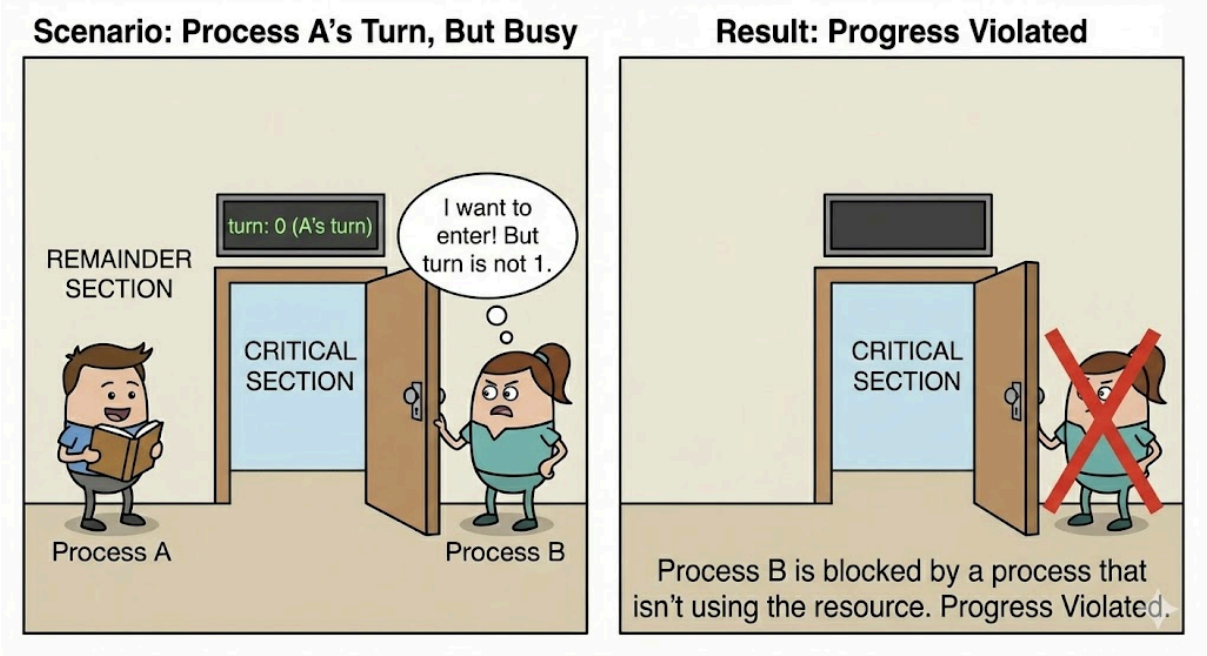
```
    // --- EXIT SECTION ---
    lock = 0; // Release lock [cite: 113]
    // ----------------------

    return NULL;
}

int main() {
    pthread_t p1, p2;
    int id1 = 0, id2 = 1;

    // Create two processes running simultaneously
    pthread_create(&p
1, NULL, process, &id1);
    pthread_create(&p2, NULL, process, &id2);

    pthread_join(p1, NULL);
    pthread_join(p2, NULL);

    return 0;
}
```

Output:

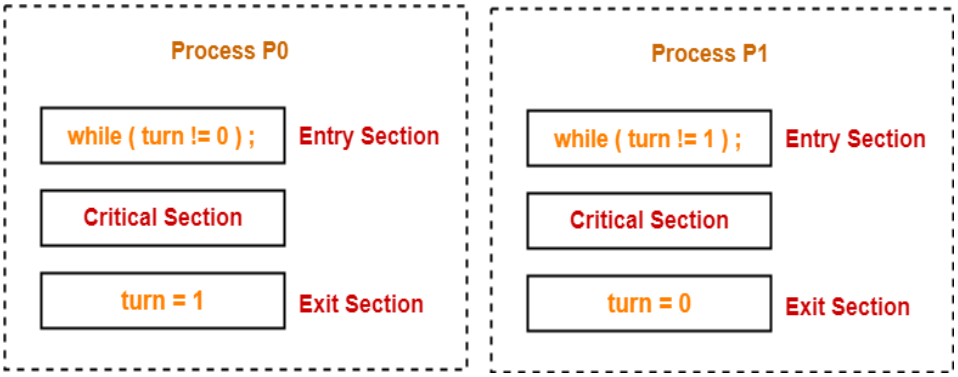
```
Process 0: Checking lock...
Process 1: Checking lock...
Process 0: ENTERED Critical Section!
Process 1: ENTERED Critical Section!
Process 0: LEAVING Critical Section.
Process 1: LEAVING Critical Section.
```

Strict Alternation (The "Turn Variable")

- **The Idea:**
  - We use a variable turn to assign strict turns.
  - If turn == 0, it is **Process A's** turn. Process B must wait.
  - If turn == 1, it is **Process B's** turn. Process A must wait.
  - They pass the turn back and forth like a ping-pong ball.



- **Works For: Mutual Exclusion** (It actually prevents two processes from entering at once).
- **The Failure (The "Vacation" Problem):**
  - Strict turns force processes to depend on each other.
  - **Scenario:**
    - It is Process A's turn (turn = 0).
    - Process A is busy doing other things (Remainder Section) and doesn't want to enter the Critical Section.
    - Process B *really* wants to enter, but it is blocked simply because turn is not 1.
  - Process B is stuck waiting for a process that isn't even using the resource.
- **Violates: Progress** (An empty room is blocked by someone who isn't using it).



Mutual Exclusion ✅ - Only one process can enter the critical section at a time.

Implementation Code:

```c
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

// Shared Turn Variable (0 = Process 0's turn, 1 = Process 1's turn)
int turn = 0;

void* process_0(void* arg) {
   while (1) {
       // --- ENTRY SECTION ---
       // Wait while it is NOT my turn (turn ≠ 0)
       while (turn ≠ 0);

       // --- CRITICAL SECTION ---
       printf("Process 0: ENTERED Critical Section.\n");
       sleep(1); // Simulate work
       printf("Process 0: LEAVING Critical Section.\n");
       // ————————————————

       // --- EXIT SECTION ---
       // Give turn to Process 1
       turn = 1;
```

```
        printf("Process 0: Passed turn to Process 1.\n");
        // ----------------------

        // Remainder Section (simulate time outside CS)
        sleep(1);
    }
    return NULL;
}

void* process_1(void* arg) {
    while (1) {
        // --- ENTRY SECTION ---
        // Wait while it is NOT my turn (turn ≠ 1)
        while (turn ≠ 1);

        // --- CRITICAL SECTION ---
        printf("Process 1: ENTERED Critical Section.\n");
        sleep(1); // Simulate work
        printf("Process 1: LEAVING Critical Section.\n");
        // ------------------------

        // --- EXIT SECTION ---
        // Give turn to Process 0
        turn = 0;
        printf("Process 1: Passed turn to Process 0.\n");
        // ----------------------

        // Remainder Section
        sleep(1);
    }
    return NULL;
}

int main() {
    pthread_t p0, p1;
```

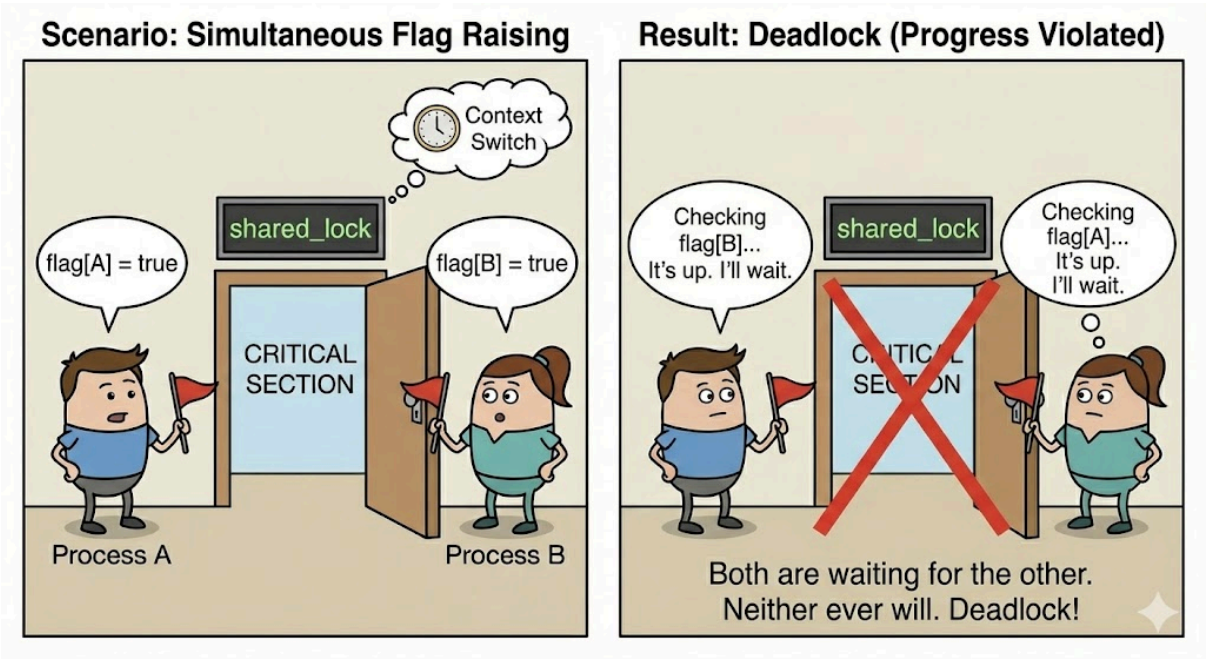
```
    // Create the two processes
    pthread_create(&p0, NULL, process_0, NULL);
    pthread_create(&p1, NULL, process_1, NULL);

    pthread_join(p0, NULL);
    pthread_join(p1, NULL);

    return 0;
}
```

Output:

```
Time limit exceeded (wall clock)
```

Individual Flags (The "Polite Standoff")

- **The Idea:**
  - Instead of one turn variable, each process has its own flag.
  - Process A sets flag[A] = true to say "I want to enter."
  - Process B sets flag[B] = true to say "I want to enter."
  - Before entering, you check if the *other* person's flag is raised. If yes, you wait.



- **The Failure (Deadlock):**
  - **Scenario:**
    1. Process A raises its flag: "I want to enter."
    2. *Context Switch.*
    3. Process B raises its flag: "I want to enter."
    4. Process B checks A's flag. It is up. B waits.
    5. Process A checks B's flag. It is up. A waits.
  - Both are waiting for the other to lower their flag. Neither ever will.
- **Result: Deadlock** (They wait forever).
- **Violates: Progress**.

Summary Table:

Attempt	**Fails Because...**	**Rule Violated**
Simple Lock	Timing (Race Condition)	**Mutual Exclusion**
Strict Alternation	Forced Turns	**Progress**
Individual Flags	Deadlock	**Progress**

Implementation Code:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
#include <stdbool.h>

// Shared Flags: flag[i] = true means "Process i wants to enter"
bool flag[2] = {false, false};

void* process_0(void* arg) {
   while (1) {
       printf("Process 0: I am interested.\n");

       // --- ENTRY SECTION ---
       // 1. Raise my flag (Signal interest)
```

```
        flag[0] = true;

        // ARTIFICIAL DELAY to force Deadlock (Simulate Context Switch
here)
        // If P1 runs right now, it will ALSO raise its flag.
        sleep(1);

        // 2. Wait if the OTHER process is also interested
        printf("Process 0: Checking if Process 1 is interested...\n");
        while (flag[1] == true);
        // ————————————————

        // --- CRITICAL SECTION ---
        printf("Process 0: >>> INSIDE Critical Section <<<\n");
        sleep(1);
        // ————————————————————

        // --- EXIT SECTION ---
        // Lower my flag (I am done)
        flag[0] = false;
        printf("Process 0: Flag lowered.\n");
        // ————————————————

        sleep(1); // Remainder section
    }
    return NULL;
}

void* process_1(void* arg) {
    while (1) {
        printf("Process 1: I am interested.\n");

        // --- ENTRY SECTION ---
        // 1. Raise my flag
        flag[1] = true;
```

```
        // 2. Wait if the OTHER process is also interested
        printf("Process 1: Checking if Process 0 is interested...\n");
        while (flag[0] == true);
        // ————————————————————

        // --- CRITICAL SECTION ---
        printf("Process 1: >>> INSIDE Critical Section <<<\n");
        sleep(1);
        // ————————————————————————

        // --- EXIT SECTION ---
        flag[1] = false;
        printf("Process 1: Flag lowered.\n");
        // ————————————————————

        sleep(1); // Remainder section
    }
    return NULL;
}

int main() {
    pthread_t p0, p1;

    // Create the two processes
    pthread_create(&p0, NULL, process_0, NULL);
    pthread_create(&p1, NULL, process_1, NULL);

    pthread_join(p0, NULL);
    pthread_join(p1, NULL);

    return 0;
}
```

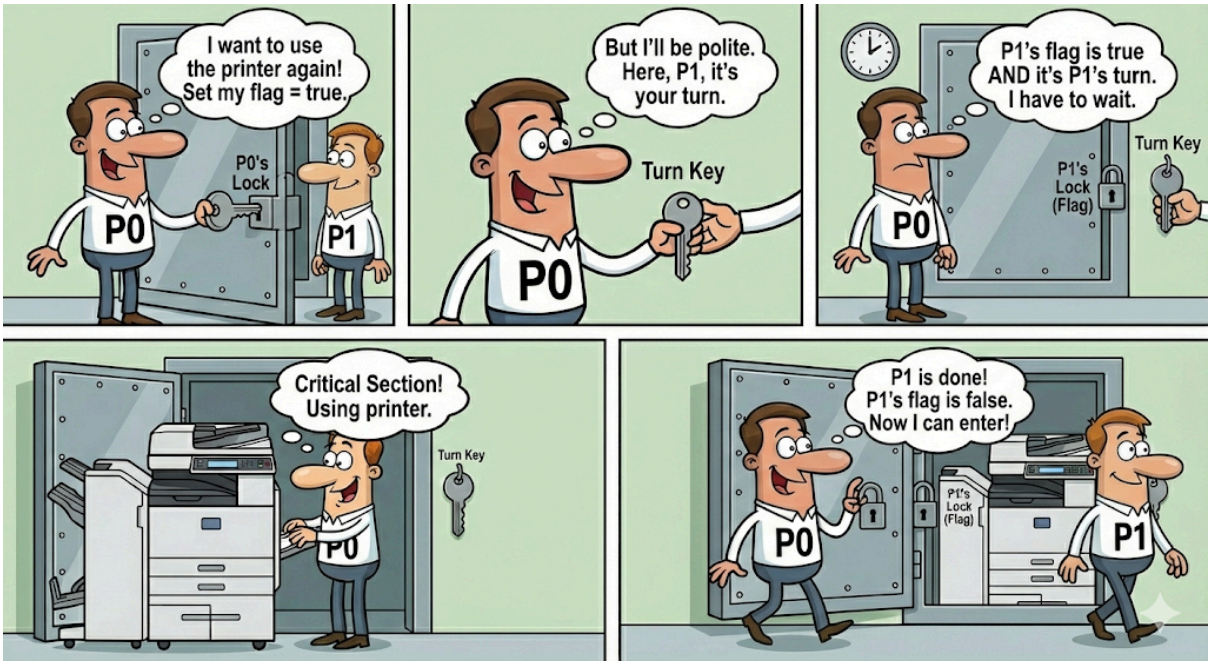
Output:

```
Time limit exceeded (wall clock)
```

Peterson’s Algorithm

The Goal

We have two processes (let's call them **P0** and **P1**) that share a resource (like a printer or a bank account). We need to ensure that only one of them accesses the resource at a time, but we also want to avoid deadlocks (where everyone waits) and starvation (where one person waits forever).



The Variables (The shared memory)

Peterson’s solution relies on two shared pieces of information available to both processes:

1. `boolean flag[2]` (The "I want it" array)
    - `flag[0] = true`: Means Process 0 wants to enter the Critical Section.
    - `flag[1] = true`: Means Process 1 wants to enter the Critical Section.
    - *Initial state:* Both are `false`.
2. `int turn` (The "Tie-Breaker")
    - This variable stores the ID of the process whose turn it is to go **if there is a conflict**.

- It can be either `0` or `1`.

Detailed Proof of Requirements

We must rigorously prove the "Holy Trinity" of synchronization.

1. Mutual Exclusion (Safety)

- **Requirement:** Only one process in CS at a time.
- **Proof:** If both P0 and P1 are in the CS, it implies that both passed the `while` loop.
  - For P0 to pass, `turn` must NOT be `1` (or `flag[1]` false).
  - For P1 to pass, `turn` must NOT be `0` (or `flag[0]` false).
  - Since both are interested, `flag[0]` and `flag[1]` are both true.
  - This implies `turn` must be both NOT 1 and NOT 0 simultaneously. This is impossible. `turn` is a single variable; it holds one value. Therefore, Mutual Exclusion holds.

2. Progress (Vitality)

- **Requirement:** If the CS is empty, the interested process enters. If both want to enter, the decision isn't postponed forever.
- **Proof:**
  - If P1 is not ready, `flag[1]` is false, so P0 enters immediately.
  - If both are ready, `turn` determines the winner immediately. There is no case where *both* are stuck waiting for the other (Deadlock) because `turn` will favor one of them.

3. Bounded Waiting (Fairness)

- **Requirement:** P1 cannot cut in line ahead of P0 forever.
- **Proof:**
  - Suppose P1 is waiting in the loop while P0 is inside the CS.
  - When P0 leaves, it runs: `flag[0] = false`.
  - This immediately breaks P1's loop condition (`flag[0]` is no longer true). P1 enters.
  - Even if P0 is super fast and tries to re-enter immediately, P0 will set `turn = 1` (giving the turn back to P1).
  - **Conclusion:** P0 cannot re-enter until P1 has had a chance. The maximum wait is **1 turn**.

Limitations (Why don't we use it everywhere?)

While Peterson's Algorithm is a perfect *logical* solution, it has issues on modern hardware:

1. **Two Processes Only:** It is complex to generalize this for N processes.
2. **Modern CPU Architecture:** Modern CPUs reorder instructions for speed. If the CPU moves "Set Flag" to happen *after* "Check Turn," the logic breaks. We

need special hardware instructions (Memory Barriers) to make Peterson's work on modern chips.

Implementation Code:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
#include <stdbool.h>

// Shared Variables
// volatile is used to prevent the compiler from optimizing away variable checks
volatile bool flag[2] = {false, false}; // Indicates if a process is ready to enter [cite: 302]
volatile int turn = 0; // Indicates whose turn it is [cite: 300]

void* process_0(void* arg) {
   while (1) {
       // --- ENTRY SECTION ---
       // 1. Indicate that Process 0 wants to enter the critical section
       flag[0] = true; // [cite: 308]

       // 2. Give priority to process P1 (The Tie-Breaker)
       turn = 1; // [cite: 309]

       // 3. Busy wait while P1 is interested AND it is P1's turn
       // "Humble behavior": If P1 wants in and I gave them the turn, I wait.
       while (flag[1] == true && turn == 1); // [cite: 310]
       // ————————————————————

       // --- CRITICAL SECTION ---
       printf("Process 0: >>> INSIDE Critical Section <<<\n");
       sleep(1); // Simulate work
       printf("Process 0: Leaving Critical Section.\n");
       // ————————————————————
```

```
        // --- EXIT SECTION ---
        // Indicate that Process 0 has exited the critical section
        flag[0] = false; // [cite: 317]
        // ----------------------

        sleep(1); // Remainder Section
    }
    return NULL;
}

void* process_1(void* arg) {
    while (1) {
        // --- ENTRY SECTION ---
        // 1. Indicate that Process 1 wants to enter the critical
section
        flag[1] = true; // [cite: 312]

        // 2. Give priority to process P0 (The Tie-Breaker)
        turn = 0; // [cite: 313]

        // 3. Busy wait while P0 is interested AND it is P0's turn
        while (flag[0] == true && turn == 0); // [cite: 314]
        // ----------------------

        // --- CRITICAL SECTION ---
        printf("Process 1: >>> INSIDE Critical Section <<<\n");
        sleep(1); // Simulate work
        printf("Process 1: Leaving Critical Section.\n");
        // ------------------------

        // --- EXIT SECTION ---
        // Indicate that Process 1 has exited the critical section
        flag[1] = false; // [cite: 318]
        // ----------------------
```

```
        sleep(1); // Remainder Section
    }
    return NULL;
}

int main() {
    pthread_t p0, p1;

    // Create the two processes
    pthread_create(&p0, NULL, process_0, NULL);
    pthread_create(&p1, NULL, process_1, NULL);

    pthread_join(p0, NULL);
    pthread_join(p1, NULL);

    return 0;
}
```

Output:

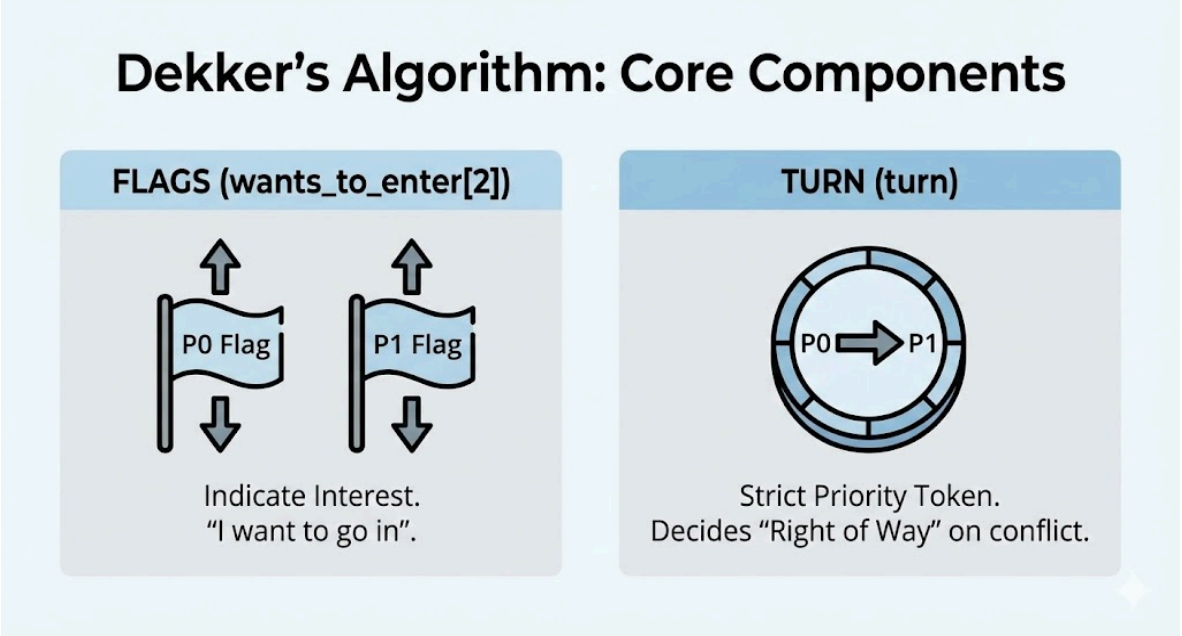
```
Process 0: >>> INSIDE Critical Section <<<
Process 0: Leaving Critical Section.
Process 1: >>> INSIDE Critical Section <<<
Process 1: Leaving Critical Section.
Process 0: >>> INSIDE Critical Section <<<
Process 0: Leaving Critical Section.
Process 1: >>> INSIDE Critical Section <<<
Process 1: Leaving Critical Section.
Process 0: >>> INSIDE Critical Section <<<
Process 0: Leaving Critical Section.
Process 1: >>> INSIDE Critical Section <<<
Process 1: Leaving Critical Section.
Process 0: >>> INSIDE Critical Section <<<
Process 0: Leaving Critical Section.
```

.
.
.
So on….(it will run continuously and give us TLE)

Dekker’s Algorithm (The First Correct Solution)

Historical Context:

Before Peterson (1981), there was Dekker (1960s). This was the first-ever correct software solution to the Critical Section problem for two processes. It proved that synchronization was possible without special hardware, though the logic is a bit more complex (and messy) than Peterson's.



Implementation Code:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
#include <stdbool.h>

#define N 5 // Number of processes (threads)

// Shared Variables
// volatile ensures the compiler doesn't cache these in registers
volatile bool choosing[N]; // Is process 'i' currently taking a
number? [cite: 341]
```

```
volatile int number[N];    // The ticket number for process 'i'
[cite: 342]

// Helper function to find the maximum ticket number currently held
int get_max_number() {
   int max = 0;
   for (int i = 0; i < N; i++) {
       if (number[i] > max) {
           max = number[i];
       }
   }
   return max;
}

void* process(void* arg) {
   int i = *(int*)arg; // My Process ID

   while (1) {
       // --- ENTRY SECTION ---

       // Step 1: Taking a ticket [cite: 345]
       choosing[i] = true;             // "I am standing at the
ticket machine"
       number[i] = get_max_number() + 1; // Take a number 1 higher
than everyone else
       choosing[i] = false;            // "I have my ticket"

       // Step 2: Wait for my turn [cite: 351]
       for (int j = 0; j < N; j++) {
           // Wait if process 'j' is currently grabbing a ticket (to
avoid race conditions)
           while (choosing[j]);

           // Wait if process 'j' has a smaller ticket number (higher
priority)
           // Or if we have the same ticket number but 'j' has a
```

```
smaller process ID (Tie-breaker) [cite: 353]
            while (number[j] != 0 &&
                   (number[j] < number[i] || (number[j] == number[i] &&
j < i)));
        }
        // ----------------------

        // --- CRITICAL SECTION ---
        printf("Process %d (Ticket %d): >>> INSIDE Critical Section
<<<\n", i, number[i]);
        sleep(1); // Simulate work
        printf("Process %d: Leaving Critical Section.\n", i);
        // ------------------------

        // --- EXIT SECTION ---
        number[i] = 0; // Drop my ticket (I am done) [cite: 356]
        // ----------------------

        sleep(1); // Remainder Section
    }
    return NULL;
}

int main() {
    pthread_t threads[N];
    int process_ids[N];

    // Create N threads
    for (int i = 0; i < N; i++) {
        process_ids[i] = i;
        pthread_create(&threads[i], NULL, process, &process_ids[i]);
    }

    // Join threads (In this infinite loop example, we won't reach
here)
    for (int i = 0; i < N; i++) {
```

```
        pthread_join(threads[i], NULL);
    }

    return 0;
}
```

Output:

```
Process 0 (Ticket 1): >>> INSIDE Critical Section <<<
Process 0: Leaving Critical Section.
Process 1 (Ticket 2): >>> INSIDE Critical Section <<<
Process 1: Leaving Critical Section.
Process 2 (Ticket 3): >>> INSIDE Critical Section <<<
Process 2: Leaving Critical Section.
Process 3 (Ticket 4): >>> INSIDE Critical Section <<<
Process 3: Leaving Critical Section.
Process 4 (Ticket 5): >>> INSIDE Critical Section <<<
Process 4: Leaving Critical Section.
Process 0 (Ticket 6): >>> INSIDE Critical Section <<<
Process 0: Leaving Critical Section.
Process 1 (Ticket 7): >>> INSIDE Critical Section <<<
Process 1: Leaving Critical Section.
Process 2 (Ticket 8): >>> INSIDE Critical Section <<<
Process 2: Leaving Critical Section.
```

.
.
.
So on….(it will run continuously and give us TLE)

Key Features

Dekker's algorithm uses the same ingredients as Peterson’s but mixes them differently.

1. **Flags (wants_to_enter[2]):**
    - Used to indicate interest ("I want to go in").
2. **Turn (turn):**
    - Used as a strict priority token. If both want to enter, the turn variable decides who has the "Right of Way."
3. **Explicit Back-Off Mechanism (The Unique Part):**
    - This is the main difference. In Peterson's, if you wait, you keep your flag up. In Dekker's, if it's not your turn, you **lower your flag** (temporarily give up) to prevent deadlocks, wait for your turn, and then raise it again.

Execution Flow (The "Polite Hallway" Logic)

Imagine two people (P0 and P1) trying to pass each other in a narrow hallway.

The Algorithm Steps for Process 0:

1. **Declare Interest:**
    - P0 raises its flag (flag[0] = true).
2. **Check for Conflict:**
    - P0 looks at P1. Is P1's flag also up?
    - **If NO:** P0 enters immediately.
    - **If YES (Conflict):** We enter the "Tie-Breaker" zone.
3. **The Tie-Breaker (Check Turn):**
    - P0 checks: "Is it my turn?"
    - **If it IS P0's turn:** P0 waits for P1 to eventually lower their flag (since P1 is polite and will see it's P0's turn).
    - **If it is P1's turn (The Back-Off):**
        1. **P0 lowers its flag** (flag[0] = false). *"I see it is your turn, so I will stop insisting."*
        2. **P0 waits** until turn becomes 0.
        3. **P0 raises its flag again** (flag[0] = true) and tries to enter.

Why the Back-Off matters:

By lowering the flag, P0 breaks the potential deadlock. It tells P1, "Go ahead, I'm not competing with you right now."

Properties (The Report Card)

Like Peterson's, Dekker's algorithm passes all the tests, but with more effort.

- **Mutual Exclusion (Safety) ✔**
    - Guaranteed because a process only enters if the other process's flag is down OR if the turn is clearly theirs. They can never both enter at once.
- **Progress (No Deadlock) ✔**
    - The "Back-Off" ensures they don't get stuck staring at each other. Ideally, the one whose turn it is keeps their flag up, and the other drops theirs, allowing the winner to proceed.
- **Bounded Waiting (Fairness) ✔**

- The turn variable alternates. Once P1 finishes, it swaps turn to P0, ensuring P0 is the next one allowed to enter.

Comparison: Dekker vs. Peterson

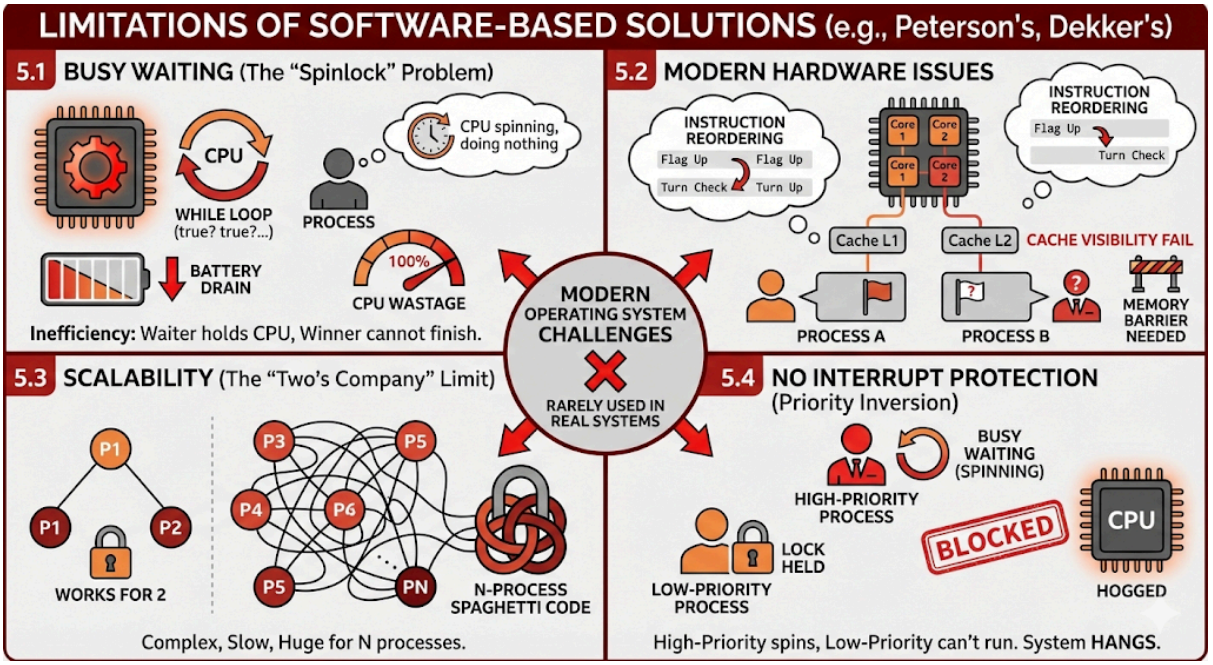
Feature	Dekker's Algorithm	Peterson's Algorithm
Logic	Complex. Requires nested if statements and flag toggling.	Simple. Just one while loop logic.
Action on Conflict	**Backs off** (Drops flag, waits, raises flag).	**Busy Waits** (Keeps flag up, but loops politely).
Significance	The **first** correct solution (Historical).	The **best** software solution (Practical/Academic).

Limitations of Software-Based Solutions

While algorithms like Peterson's and Dekker's are clever, they are rarely used in real modern operating systems. Here is why they are problematic.

Busy Waiting (The "Spinlock" Problem)

- **The Problem:** When a process waits (like in the while loop), it doesn't actually stop working. It stays on the CPU, spinning in circles, checking the condition over and over again (true? true? true?).
- **The Cost:**
  - **CPU Wastage:** The CPU is running at 100% speed doing nothing useful.



- **Battery Drain:** On laptops and phones, this kills the battery quickly.
- **Inefficiency:** On a single-core system, if the "Waiter" holds the CPU, the "Winner" (who is currently in the CS) cannot run to finish and release the lock. It's self-defeating.

Modern Hardware Issues

- **The Problem:** Modern CPUs are too smart for these old algorithms.
- **Instruction Reordering:** To run faster, the CPU might execute line 2 before line 1 if it thinks they aren't related. In synchronization, the *order* is everything. If the CPU reorders the "Flag Up" and "Turn Check" steps, the lock fails.
- **Cache Visibility:** If Process A raises its flag in its local cache, Process B (on a different core) might not see it immediately.
- **The Fix:** You need special "Memory Barrier" instructions to force the CPU to behave, which complicates the "pure software" idea.

Scalability (The "Two's Company" Limit)

- **The Problem:** Peterson's and Dekker's are hard-coded for exactly **two** processes.
- **The Reality:** Real systems have dozens or hundreds of processes. Rewriting these algorithms for N processes makes them incredibly huge, slow, and complex.

No Interrupt Protection (Priority Inversion)

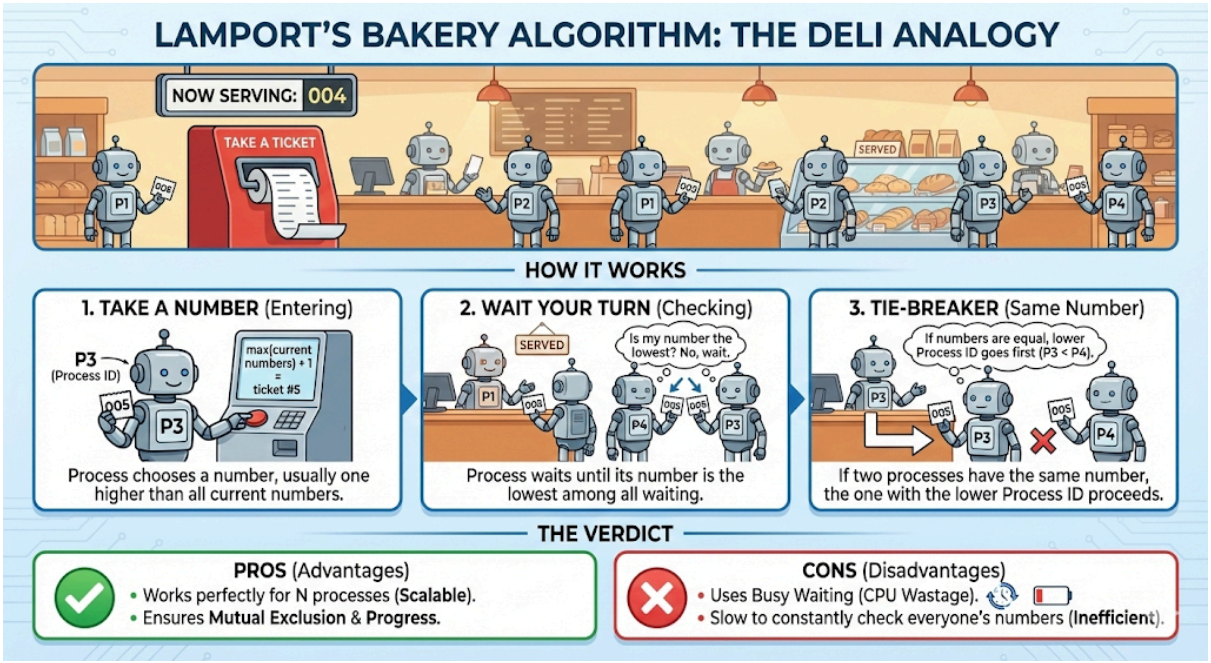
- **The Problem:** What if a Low-Priority process grabs the lock, and then a High-Priority process needs it?
- **Priority Inversion:** The High-Priority process sits there "Busy Waiting" (spinning), wasting CPU time, while the Low-Priority process (which holds the lock) cannot run because the High-Priority one is hogging the CPU. The system hangs.

n-Process Generalizations

Since Peterson's only works for two, we needed a way to handle N processes. The most famous attempt is:

Lamport's Bakery Algorithm

- **The Analogy:** Think of a busy bakery or a deli counter.



- **How it works:**
  - **Take a Number:** When a process wants to enter, it "takes a ticket" (chooses a number). The number is usually max(all_current_numbers) + 1.
  - **Wait your turn:** The process waits until its number is the lowest one in the room.
  - **Tie-Breaker:** If two processes accidentally pick the same number, the one with the lower Process ID goes first.
- **The Verdict:**
  - **Pros:** It works perfectly for N processes.
  - **Cons:** It still uses Busy Waiting and is very slow to check everyone's ticket numbers constantly.

Implementation Code:

```c
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
#include <stdbool.h>

#define N 5 // Number of processes (Customer threads)
```

```
// Shared Variables (Slide 33)
// choosing[i]: true when process 'i' is currently taking a ticket
volatile bool choosing[N];
// number[i]: The ticket number held by process 'i' (0 = not
interested)
volatile int number[N];

// Helper: Find the maximum ticket number currently in use
int get_max_ticket() {
   int max = 0;
   for (int i = 0; i < N; i++) {
       if (number[i] > max) {
           max = number[i];
       }
   }
   return max;
}

void* process(void* arg) {
   int i = *(int*)arg; // My Process ID

   while (1) {
       // --- ENTRY SECTION (Slide 34) ---

       // Step 1: Taking a Ticket
       // "I am standing at the ticket machine..."
       choosing[i] = true;

       // "I take a number one greater than the max currently held"
       number[i] = get_max_ticket() + 1;

       // "I have my ticket."
       choosing[i] = false;

       // Step 2: Wait for my Turn
```

```
        // Check against every other process 'j'
        for (int j = 0; j < N; j++) {

            // Wait if process 'j' is currently grabbing a ticket.
            // Why? Because if they are choosing, they might pick a
smaller number than us
            // (race condition), so we must wait to see what they get.
            while (choosing[j]);

            // Lexicographical Comparison (Slide 35):
            // Wait if:
            // 1. Process 'j' is interested (number[j] ≠ 0) AND
            // 2. Process 'j' has a smaller ticket number than me OR
            // 3. We have the same ticket, but 'j' has a smaller ID
(Tie-breaker)
            while (number[j] ≠ 0 &&
                   (number[j] < number[i] || (number[j] == number[i] &&
j < i)));
        }
        // ————————————————————————

        // --- CRITICAL SECTION ---
        printf("Process %d (Ticket %d): >>> SERVED (Inside CS) <<<\n",
i, number[i]);
        sleep(1); // Simulate using the resource
        printf("Process %d: LEAVING CS.\n", i);
        // ————————————————————

        // --- EXIT SECTION (Slide 34) ---
        // "Throw away ticket" (Set to 0)
        number[i] = 0;
        // ————————————————————————

        sleep(2); // Remainder Section
    }
    return NULL;
```

```
}

int main() {
   pthread_t threads[N];
   int process_ids[N];

   // Initialize shared arrays
   for(int i=0; i<N; i++) {
       choosing[i] = false;
       number[i] = 0;
   }

   // Create N threads (processes)
   for (int i = 0; i < N; i++) {
       process_ids[i] = i;
       pthread_create(&threads[i], NULL, process, &process_ids[i]);
   }

   // Join threads
   for (int i = 0; i < N; i++) {
       pthread_join(threads[i], NULL);
   }

   return 0;
}
```

Output:

```
Process 0 (Ticket 1): >>> INSIDE Critical Section <<<
Process 0: Leaving Critical Section.
Process 1 (Ticket 2): >>> INSIDE Critical Section <<<
Process 1: Leaving Critical Section.
Process 2 (Ticket 3): >>> INSIDE Critical Section <<<
Process 2: Leaving Critical Section.
Process 3 (Ticket 4): >>> INSIDE Critical Section <<<
Process 3: Leaving Critical Section.
Process 4 (Ticket 5): >>> INSIDE Critical Section <<<
Process 4: Leaving Critical Section.
Process 0 (Ticket 6): >>> INSIDE Critical Section <<<
Process 0: Leaving Critical Section.
Process 1 (Ticket 7): >>> INSIDE Critical Section <<<
Process 1: Leaving Critical Section.
Process 2 (Ticket 8): >>> INSIDE Critical Section <<<
Process 2: Leaving Critical Section.
```

.
.
.
So on….(it will run continuously and give us TLE)

Why Software Solutions Are Not Used Today

If these algorithms are so smart, why don't Windows or Linux use them?

1. Hardware is Better

- Modern CPUs provide **Atomic Instructions** (hardware magic).
- Instructions like Test-and-Set or Compare-and-Swap can check and lock a variable in **one single, unbreakable CPU cycle**. It is faster and safer than any software loop.

2. OS Tools are More Efficient

- Operating Systems provide **Mutexes** and **Semaphores**.
- Unlike software loops that "Spin" (Busy Wait), these tools put the waiting process to **Sleep**. The process leaves the CPU, saving battery and allowing others to work. It only wakes up when the resource is free.

3. The Role of Software Algorithms

- They are **Conceptual Foundations**. We study them to understand *why* synchronization is hard and to learn the logic of "Flags" and "Turns," but we do not write them in production code.